

Bonus: High-Throughput CDC Lakehouse Architecture

Domain: Distributed Data Engineering & Cloud Lakehouse Platforms | Scale: 50M+ mutating rows/day

1. Problem Statement & Operational Context

Modern cloud analytical platforms demand sub-15-minute data freshness from operational OLTP databases (PostgreSQL/MySQL). Standard data lakes struggle when continuously mutating accounting and transaction records are streamed in real time. The objective is to design an enterprise-grade Change Data Capture (CDC) ingestion pipeline into a Lakehouse table (Delta Lake / Apache Iceberg) that guarantees strictly deterministic historical auditing without destabilizing downstream query performance.

Strict Real-World Constraints Designed to Prevent Generic AI Answers:

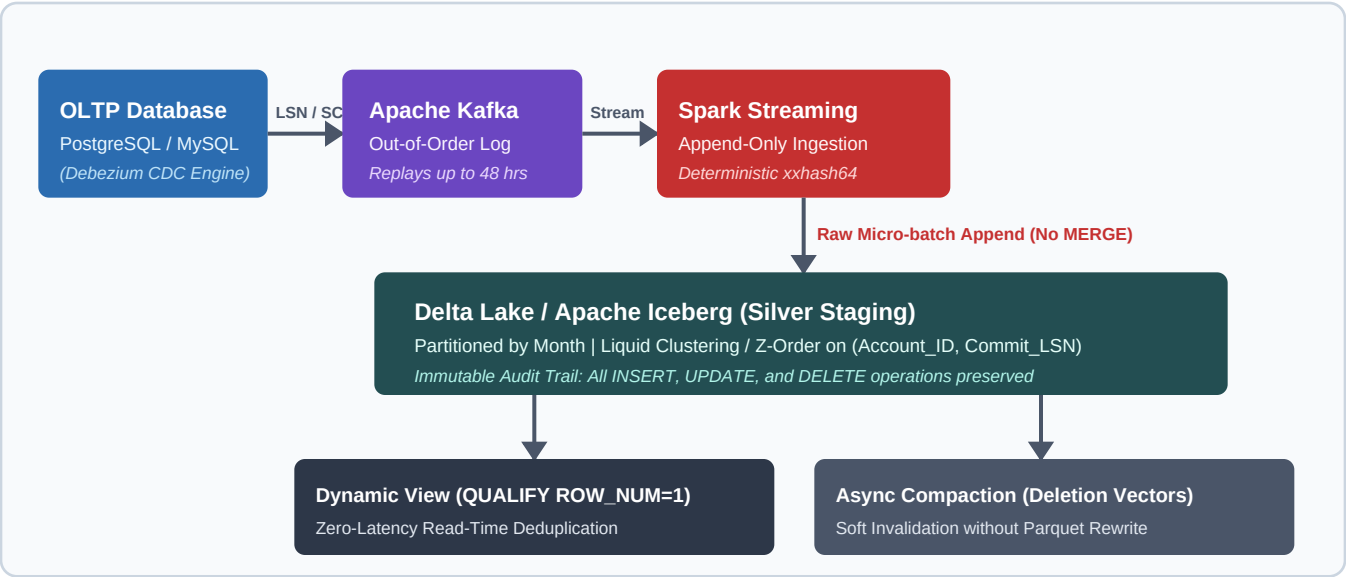
- 1. Absolute Prohibition of Continuous MERGE INTO:** Target tables exceed 4 billion rows (multiple terabytes). Running continuous standard MERGE INTO ... WHEN MATCHED is banned due to extreme Parquet file-rewrite amplification (Copy-on-Write) and concurrent metadata commit collisions with analytical readers.
- 2. Out-of-Order Events & Replays:** Network partitions and Kafka consumer group rebalances cause events to arrive up to 48 hours late and duplicated. Simple wall-clock timestamp comparisons are invalid.
- 3. No External State Stores (Redis / RocksDB):** Cost and air-gapped security policies prohibit external key-value cache clusters for streaming deduplication.
- 4. Immutable Audit Regulations:** Regulatory compliance forbids physical purging of rows. Hard deletes must be converted into deterministic bitemporal valid-time intervals.

2. Why Generic LLM Recommendations Fail

Generic LLM Advice	Technical Root Cause Failure Under Our Constraints
'Just run MERGE INTO in every Spark micro-batch.'	Catastrophic Write Amplification. Updating a single 200-byte row forces Spark to rewrite an entire 128 MB Parquet file. For 50M updates, storage I/O and cloud compute costs explode, while readers experience read-lock failures.
'Maintain a 48-hour state store in Redis or RocksDB.'	Violates Constraint 3 & Recovery SLAs. Tracking 48 hours of high-cardinality keys in memory introduces massive infrastructure overhead and network serialization bottlenecks during cluster node recovery.
'Generate surrogate keys with monotonically_increasing_id().'	Non-Deterministic Failure. Native Spark monotonic IDs are partition-dependent. Job restarts, partition shifts, or backfill runs generate completely different IDs for the same business entity, corrupting downstream data mart joins.

3. Target Architecture: Write-Optimized Staging + Read-Time Reconciliation

The non-generic solution decouples high-speed data ingestion from row-level deduplication. Writes are strictly append-only, while deduplication is handled at query time via dynamic views and asynchronous compactions.



4. Core Pipeline Mechanics

Component	Technical Implementation	Operational Benefit
1. Append-Only Staging	Spark Streaming consumes Kafka CDC events and writes exclusively via <code>.format('delta').mode('append')</code> . Zero row merges occur in the hot path.	Micro-batches complete in 1-2 seconds with minimal CPU and zero storage write amplification.
2. Deterministic Keys	Surrogate keys are generated via <code>xxhash64(concat(biz_key, source_lsn))</code> .	Ensures 100% idempotent keys across backfills and job restarts without external key caches.
3. Deletion Vectors	Isolated background jobs mark stale rows using Delta Deletion Vectors (or Iceberg position deletes).	Invalidates superseded records via lightweight bitmap files without touching or rewriting existing Parquet blocks.
4. Analytical Serving	Consumers query a dynamic SQL view that resolves the latest record per business key on demand.	Delivers sub-second data freshness to downstream reporting engines without locking raw staging tables.

5. Production View Definition (Dynamic Read-Time Reconciliation)

```
-- Analytical Gold View: Pushes reconciliation to query execution
CREATE OR REPLACE VIEW gold_accounting_current AS
SELECT
    surrogate_key,
    account_id,
    current_balance,
    transaction_timestamp,
    source_commit_lsn
FROM silver_accounting_cdc_staging
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY account_id
    ORDER BY source_commit_lsn DESC, kafka_event_offset DESC
) = 1
AND operation_type != 'DELETE'; -- Filters soft-deleted accounts without physical purging
```